Project 17.1 — ITDesk-IT-Helpdesk-Bot-With-Human-Escalation

A practice twin of Project 17 — DeflectDesk. Same skills — RAG over a knowledge base, a deterministic **confidence gate**, and a real **human handoff** (`interrupt` + checkpointer on a LangGraph graph) — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: RAG agent on LangGraph with human-in-the-loop · **Stack:** Python, LangChain, Chroma (RAG), LangGraph (`interrupt`, checkpointer), OpenAI SDK · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

An internal IT helpdesk gets the same questions all day — "how do I connect to the VPN", "how do I install the design software", "where's the printer queue". Those are in the knowledge base. But some requests must **never** be auto-answered: "my account is locked", "I think I clicked a phishing link", "reset my admin access". Those need a real human, and a bot that confidently answers them is a security incident waiting to happen.

You are going to build **ITDesk**: a helpdesk bot that answers routine questions from the IT knowledge base and **escalates the sensitive ones to a human**.

- A **retrieve** node searches the IT KB and records the best match's distance.
- A **router** checks it against a cutoff: close enough → answer from the KB; too far → escalate.
- The **escalate** node calls `interrupt()`, pausing the graph until an IT staffer replies — and a checkpointer saves the state so the pause can last.

The one lesson of this project: same as DeflectDesk — the skill is knowing *when not to answer*. A confidence gate plus a real human handoff is exactly what keeps a helpdesk bot from auto-"resetting" an account it should never touch.

How it works, from a question to an answer or a handoff (picture the whole machine)

A question comes in. The **retrieve** node searches the KB (Chroma) and writes the closest article and its distance into state. The **router** runs the gate: `is_confident(distance, cutoff)` → "answer" or "escalate". The **answer** node replies grounded in the matched article. The **escalate** node calls `interrupt()`, which pauses the graph; an IT staffer reads the paused request, replies, and the graph **resumes** with their answer (`escalated = True`). A checkpointer saves the state so the pause survives.

That's the whole machine: **question → retrieve → confident? answer from KB : escalate (interrupt → human → resume).**

What you are building (the pieces)

- A RAG layer over the IT KB (`build_kb_store` , `best_match`).
- A deterministic `is_confident(distance, cutoff)` gate.
- A LangGraph graph (retrieve → route → answer / escalate with `interrupt`) + a checkpointer.
- A `main.py` that answers, or pauses for a human and resumes.

Inputs: an IT question. **Outputs:** an answer from the KB, or a human's reply after escalation. **Rules:** match by meaning (RAG); the gate is deterministic (testable with no model); sensitive/off-topic questions escalate and genuinely pause.

A real example to build and test against

- "How do I connect to the VPN?" (covered in the KB) → answered from the KB.
- "My account is locked, reset it" (not in the KB) → escalates: the graph pauses, an IT staffer replies, and it resumes with `escalated = True` .

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 17 / Doc08 RAG / Doc09 `interrupt` if hazy.)
2. Build the **confidence gate first** and test it with made-up distances — no model.
3. Build the RAG layer, then the graph (route on the gate; `interrupt` in the escalate node).
4. Compile with a checkpointer and test both paths: a covered question answered, a sensitive one escalated-then-resumed.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

Every assistant that touches anything sensitive — IT, finance, healthcare, legal — needs the same guardrail: answer the routine, hand off the risky. The confidence gate (a retrieval distance under a cutoff) plus a real `interrupt`-based handoff is a reusable, trustworthy pattern. Build ITDesk and you can put a safe "knows when to get a human" boundary on any knowledge-base bot.

Done when

- The bot answers only when the best KB match is under the cutoff.
- A sensitive/off-topic request escalates — it never auto-answers it.
- `is_confident` is a deterministic gate, tested with no model.
- Escalation really pauses the graph (`interrupt`) and resumes with a human reply.
- Your tests pass: the gate (fake distances) and a live covered-vs-escalated run.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.